MAVEN 安装部署文档

北京首钢自动化信息技术有限公司

2024 年 9 月

版 本 号	创 建 时 间	作　　者	受控状态	审核一	审核二
1.0	2024-09-01	刘壮			

文件修改记录

序号	修改后版本号	修改日期	修改人	修改原因/说明	修改内容

目 录

序号	主机名称	主机名	地址 网关 DNS	应用包部署位置	部署基础软件	备注
1	nexus1	Nexus1	192.168.168.11/24 192.168.168.1 192.168.168.1	/app/nginx/static	Nexus	Nexus 主私服
	nexus2	Nexus2	192.168.168.12/24 192.168.168.1 192.168.168.1	/app/nginx/static	Nexus	Nexus 从私服

1 准备

1.本地开发机：

首先声明公司内部是有自己的 nexus 仓库，但是对上传 jar 包做了限制，不能畅快的上传自己测试包依赖。于是就自己在本地搭建了一个 nexus 私服，即可以使用公司 nexus 私服仓库中的依赖，也可以上传和使用自己的测试包依赖。

2.公司搭建私服：

公司都不提供外网给项目组人员，因此就不能使用 maven 访问远程的仓库地址，所以很有必要在局域网里找一台有外网权限的机器，搭建 nexus 私服，然后开发人员连到这台私服上，这样的话就可以通过这台搭建了 nexus 私服的电脑访问

maven 的远程仓库。还有就是公司有自己的 jar 可以发布到私服上等等。

1.1 操作环境

```
Operating System:Red Hat Enterprise Linux Server 7.3 (Maipo)
Kernel: Linux 3.10.0-514.el7.x86_64
Architecture: x86-64
```

1.2 主机信息信息检查

1.2.1 网络相关检查

1) 检查确认主机名，与规划文档进行核对确认主机名是否需要修改。

检查:

```
[root@localhost /]# hostnamectl
```

```
   Static hostname: localhost.localdomain
         Icon name: computer-vm
           Chassis: vm
        Machine ID: 8c1bc3845d4146fd8d425695208eef5b
           Boot ID: f0dae4a3e9d145a2b924e4f73699e0d9
    Virtualization: vmware
  Operating System: Red Hat Enterprise Linux Server 7.3 (Maipo)
       CPE OS Name: cpe:/o:redhat:enterprise_linux:7.3:GA:server
            Kernel: Linux 3.10.0-514.el7.x86_64
      Architecture: x86-64
```

修改:

```
[root@localhost /]# hostnamectl --static set-hostname sitpp4

[root@sitpp1 ~]# hostnamectl
```

```
   Static hostname: sitpp1
```

```
        Icon name: computer-vm
          Chassis: vm
       Machine ID: 8c1bc3845d4146fd8d425695208eef5b
          Boot ID: 9a3c0ae828e04ad894851d3594af7c59
   Virtualization: vmware
 Operating System: Red Hat Enterprise Linux Server 7.3 (Maipo)
      CPE OS Name: cpe:/o:redhat:enterprise_linux:7.3:GA:server
           Kernel: Linux 3.10.0-514.el7.x86_64
     Architecture: x86-64      Architecture: x86-64
```

2) 执行以下命令确认是否删除虚拟网络。

检查方法如下：

```
[root@localhost /]# ip addr list
```

```
3: virbr0: <NO-CARRIER,BROADCAST,MULTICAST,UP> mtu 1500 qdisc noqueue state DOWN qlen
1000
    link/ether 52:54:00:7e:0d:65 brd ff:ff:ff:ff:ff:ff
    inet 192.168.122.1/24 brd 192.168.122.255 scope global virbr0
       valid_lft forever preferred_lft forever
4: virbr0-nic: <BROADCAST,MULTICAST> mtu 1500 qdisc pfifo_fast master virbr0 state
DOWN qlen 1000
    link/ether 52:54:00:7e:0d:65 brd ff:ff:ff:ff:ff:ff
```

删除虚拟网络

```
[root@localhost /]# virsh net-list

[root@localhost /]# virsh net-destroy default

[root@localhost /]# virsh net-undefine default

[root@localhost /]# systemctl restart libvirtd.service

[root@localhost /]# virsh net-list

[root@localhost /]# systemctl restart network
```

ifconfig virbr0 down

brctl delbr virbr0　　//删除网桥

systemctl disable libvirtd.service　　//禁用 libvirtd 服务开机自启动

确认是否删除虚拟网络

```
[root@localhost /]# ip addr list
```

```
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN qlen 1
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host
       valid_lft forever preferred_lft forever
2: ens224: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc mq state UP qlen 1000
    link/ether 00:50:56:87:56:ad brd ff:ff:ff:ff:ff:ff
    inet 192.168.147.138/24 brd 10.1.247.255 scope global ens224
       valid_lft forever preferred_lft forever
    inet6 fe80::7715:dd56:d4d:cf97/64 scope link
       valid_lft forever preferred_lft forever
    inet6 fe80::16f3:cfa6:5fc8:f654/64 scope link tentative dadfailed
       valid_lft forever preferred_lft forever
```

3)　IP 地址、DNS 信息,与规划文档进行核对确认主机名是否需要修改。

检查 DNS

```
[root@localhost /]# nmcli connection show ens192
```

```
connection.id:                          ens192 connection.uuid:
4df0d4f9-bf11-4a87-b34a-54e6d12ed426 connection.stable-id:                  --
connection.interface-name:              ens192 connection.type:

……
```

ens192 为网卡名称

设置 DNS

```
[root@localhost /]# nmcli conn modify lan1 ipv4.dns 10.1.135.135
```

```
[root@localhost /]# nmcli conn modify lan1 +ipv4.dns 10.1.135.135
```

删除 DNS

```
[root@localhost /]# nmcli conn modify lan1 -ipv4.dns 202.96.128.86
```

设置 IP

```
[root@localhost /]# nmcli connection modify lan1 ipv4.addresses 192.168.147.138/24

[root@localhost /]# nmcli connection modify lan1 +ipv4.addresses 192.168.147.138/24

[root@localhost /]# nmcli connection modify lan1 ipv4.method manual

[root@localhost /]# systemctl restart network
```

删除 IP

```
[root@localhost /]# nmcli connection modify lan1 -ipv4.addresses 192.168.147.138/24
```

4) 时钟同步

查看时区

```
# timedatectl
```

```
      Local time: Mon 2019-05-13 16:16:24 CST
  Universal time: Mon 2019-05-13 08:16:24 UTC
        RTC time: Mon 2019-05-13 08:15:59
       Time zone: Asia/Shanghai (CST, +0800)
     NTP enabled: yes
NTP synchronized: no
 RTC in local TZ: no
      DST active: n/a
```

```
# timedatectl list-timezones
```

```
……
Asia/Riyadh
Asia/Sakhalin
Asia/Samarkand
Asia/Seoul
Asia/Shanghai
```

```
Asia/Singapore
Asia/Srednekolymsk
Asia/Taipei
Asia/Tashkent
Asia/Tbilisi
Asia/Tehran
Asia/Thimphu
Asia/Tokyo
Asia/Tomsk
.....
```

设置时区

```
# timedatectl set-timezone Asia/Shanghai
```

修改时间

```
# timedatectl set-time "2015-01-21 11:50:00" (可以只修改其中一个)
```

设置时间同步

chrony 方式配置

```
[root@localhost]# yum install chrony

[root@localhost]# rpm -qa |grep chrony

[root@localhost]# cp /etc/chrony.conf{,.bak}

[root@localhost]# echo "

#内部 NTP 服务

server ntp01.gfcx.com iburst

server ntp02.gfcx.com iburst

server ntp03.gfcx.com iburst

server ntp04.gfcx.com iburst

driftfile /var/lib/chrony/drift
```

```
makestep 1.0 3

rtcsync

logdir /var/log/chrony

">/etc/chrony.conf

[root@localhost]# systemctl start chronyd.service

[root@localhost]# systemctl restart chronyd.service

[root@localhost]# systemctl status chronyd.service

[root@localhost]# systemctl enable chronyd.service

[root@localhost]# chronyc sources -v

[root@localhost]# chronyc sourcestats -v
```

5) SELINUX

```
[root@localhost /]# sed -i 's/SELINUX=enforcing/SELINUX=disabled/' /etc/selinux/config

[root@localhost /]# grep SELINUX=disabled /etc/selinux/config
```

```
SELINUX=disabled
```

```
[root@localhost /]# setenforce 0
```

6) 防火墙设置

firewall 关闭

```
[root@localhost /]# systemctl stop firewalld.service

[root@localhost /]# systemctl disable firewalld.service
```

firewall 开启

```
[root@localhost /]# systemctl start firewalld.service

[root@localhost /]# systemctl enable firewalld.service
```

开启 firewall 情况下开启 vrrp 协议，主备都运行下面的命令

```
[root@localhost /]# firewall-cmd --direct --permanent --add-rule ipv4 filter INPUT 0 -
-in-interface em1 --destination 224.0.0.18 --protocol vrrp -j ACCEPT
```

```
[root@localhost /]# firewall-cmd --reload
```

1.2.2 磁盘相关检查

确认磁盘大小与规划大小是否一致，确认分区是否一致

```
[root@localhost /]# fdisk -l
```

1.2.3 CPU、内存检查

确认 CPU、内存与规划大小是否一致，确认分区是否一致

查看 CPU 个数

```
# cat /proc/cpuinfo | grep "physical id" | uniq | wc -l
```

uniq 命令：删除重复行;wc –l 命令：统计行数

查看 CPU 核数

```
# cat /proc/cpuinfo | grep "cpu cores" | uniq
```

cpu cores : 4

查看 CPU 型号

```
# cat /proc/cpuinfo | grep 'model name'| uniq
```

model name : Intel(R) Xeon(R) CPU E5630 @ 2.53GHz

查看内存总数

```
#cat /proc/meminfo | grep MemTotal
```

MemTotal: 32941268 kB //内存 32G

1.2.4 目录相关检查

检查介质目录

```
[root@localhost /]# ll /software
```

创建介质目录

```
[root@localhost /]# mkdir /software
```

将相关介质放置该目录备用

1.3 Nexus 安装配置

2 环境搭建

2.1 资源下载

1.首先确定我们的环境安装好 maven，jdk 等必须的环境

2.这些都准备好之后，去下载最新版本的 nexus

下载地址：http://www.sonatype.org/nexus/go

2.2 解压

将下载的 nexus-3.14.0-04-win64.zip 解压到自定义目录即可。

2.3 配置

1) 端口配置，绑定地址配置

打开 zip 解压文件下的 ../nexus-3.14.0-04-win64/nexus-3.14.0-04/etc/nexus-default.properties。

nexus-default.properties | nexus.vmoptions

```
1  ## DO NOT EDIT - CUSTOMIZATIONS BELONG IN $data-dir/etc/nexus.properties
2  ##
3  # Jetty section
4  application-port=8181
5  application-host=127.0.0.1
6  nexus-args=${jetty.etc}/jetty.xml,${jetty.etc}/jetty-http.xml,${jetty.etc}/jetty-requestlog.xml
7  nexus-context-path=/
8
9  # Nexus section
10 nexus-edition=nexus-pro-edition
11 nexus-features=\
12  nexus-pro-feature
13
14 # 访问地址 application-host=0.0.0.0  http://localhost:8181/
15 # 访问地址 application-host=127.0.0.1  http://127.0.0.1:8181/
```

2) 如下属性可以自定义修改。

application-host : Nexus 服务监听的主机

application-port: Nexus 服务监听的端口，

nexus-context-path : Nexus 服务的上下文路径

通常可以不做任何修改，但个人习惯于修改 application-host 为 0.0.0.0（关于 0.0.0.0 与 127.0.0.1 的区别自行检索），我这里修改了端口和 host。

3) 内存大小配置

打开解压目录下的 ../nexus-3.14.0-04-win64/nexus-3.14.0-04/bin/nexus.vmoptions

可以在下图配置运行时的最大堆、最小堆等，可以根据个人的电脑以及需要修改，默认配置如下。

nexus-default.properties | nexus.vmoptions

```
-Xms1200M
-Xmx1200M
-XX:MaxDirectMemorySize=2G
-XX:+UnlockDiagnosticVMOptions
-XX:+UnsyncloadClass
-XX:+LogVMOutput
-XX:LogFile=../sonatype-work/nexus3/log/jvm.log
-XX:-OmitStackTraceInFastThrow
-Djava.net.preferIPv4Stack=true
-Dkaraf.home=.
-Dkaraf.base=.
-Dkaraf.etc=etc/karaf
-Djava.util.logging.config.file=etc/karaf/java.util.logging.properties
-Dkaraf.data=../sonatype-work/nexus3
-Djava.io.tmpdir=../sonatype-work/nexus3/tmp
-Dkaraf.startLocalConsole=false

```

4) nexus 注册服务

在.../nexus-3.14.0-04-win64/nexus-3.14.0-04/bin 目录下，必须以管理员身份运行 cmd ：

nexus.exe /run 命令可以启动 nexus 服务（参考官方文档)安装 nexus 本地服务来启动(推荐使用这种方式，参考官方文档),命令如下所示。

安装 nexus 服务

```
PS D:\Nexus\nexus-3.14.0-04\bin> ./nexus.exe /install        //安装 nexus 服务

PS D:\Nexus\nexus-3.14.0-04\bin> ./nexus.exe /start         //启动 nexus 服务

D:\Nexus\nexus-3.14.0-04\bin> ./nexus.exe /stop           //停止 nexus 服务
```

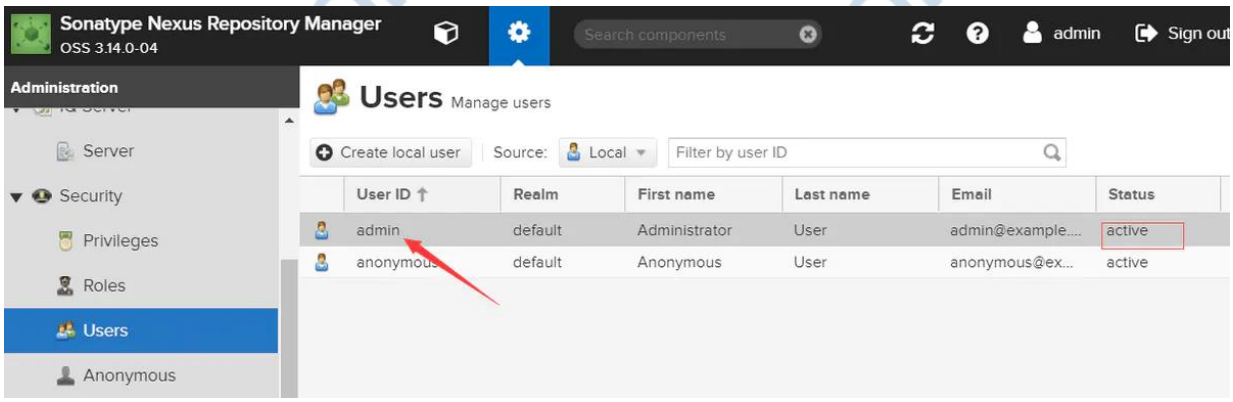
5) 登录

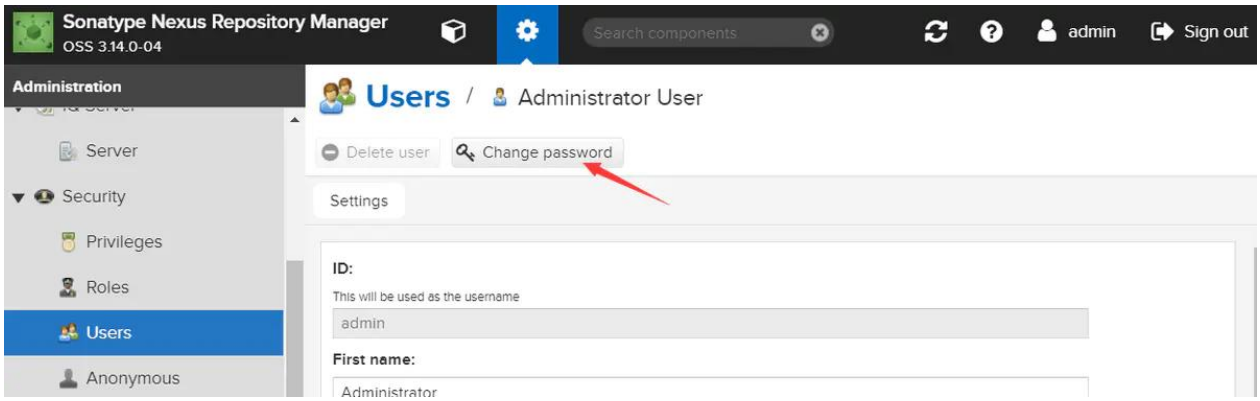
如果没有做任何端口和上下文路径的修改，直接访问 http://localhost:8081 即可。

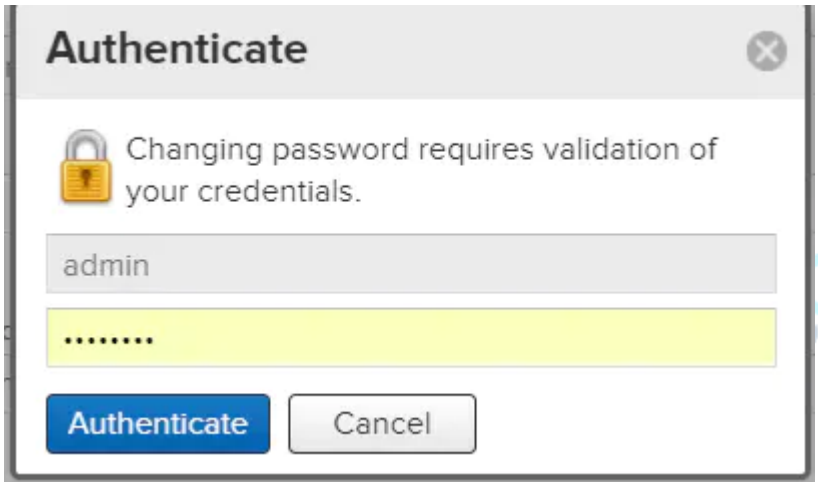
我这里更改成 http://127.0.0.1:8181

默认的用户名和密码分别是：admin/amdin123

修改密码：




Sonatype Nexus Repository Manager
OSS 3.14.0-04
Search components
admin
Sign out
Administration
Server
Security
Privileges
Roles
Users
Anonymous
Users / Administrator User
Delete user
Change password
Settings
ID:
This will be used as the username
admin
First name:
Administrator


Authenticate
Changing password requires validation of your credentials.
admin
Authenticate
Cancel


Change Password
New password:
Confirm password:
Change password
Cancel

3 功能配置

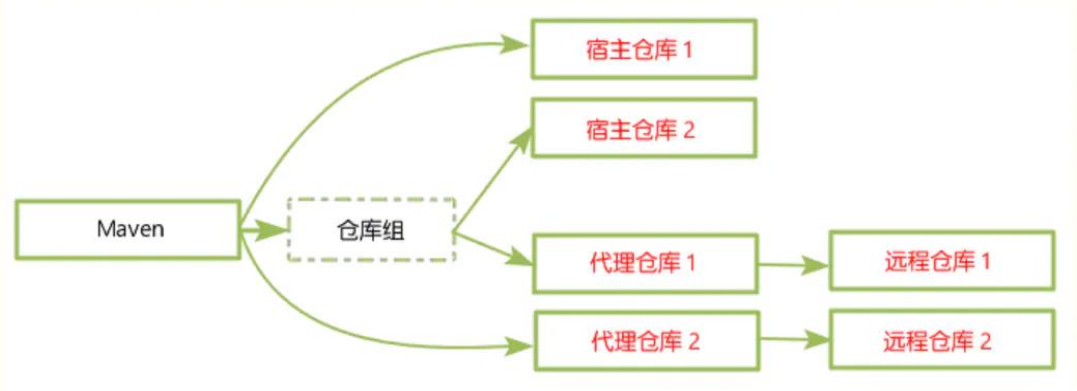
3.1 仓库类型

默认安装有以下这几个仓库，在控制台也可以修改远程仓库的地址，第三方仓库等。

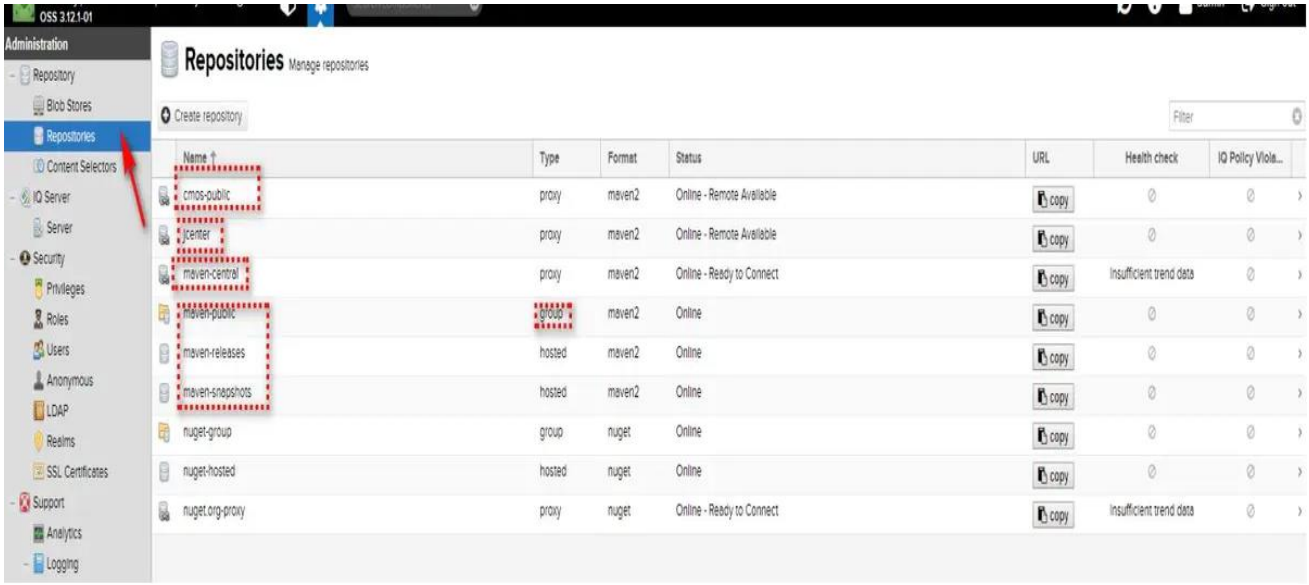
仓库名	作用
hosted（宿主仓库库）	存放本公司开发的jar包（正式版本、测试版本）
proxy（代理仓库）	代理中央仓库、Apache下测试版本的jar包
group（组仓库）	使用时连接组仓库，包含Hosted（宿主仓库）和Proxy（代理仓库）
virtual (虚拟仓库)	基本用不到，重点关注上面三个仓库的使用

Nexus 的仓库分为这么几类：

- hosted 宿主仓库：主要用于部署无法从公共仓库获取的构件（如 oracle 的 JDBC 驱动）以及自己或第三方的项目构件；
- proxy 代理仓库：代理公共的远程仓库；
- virtual 虚拟仓库：用于适配 Maven 1；
- group 仓库组：Nexus 通过仓库组的概念统一管理多个仓库，这样我们在项目中直接请求仓库组即可请求到仓库组管理的多个仓库。



3.2 分组仓库

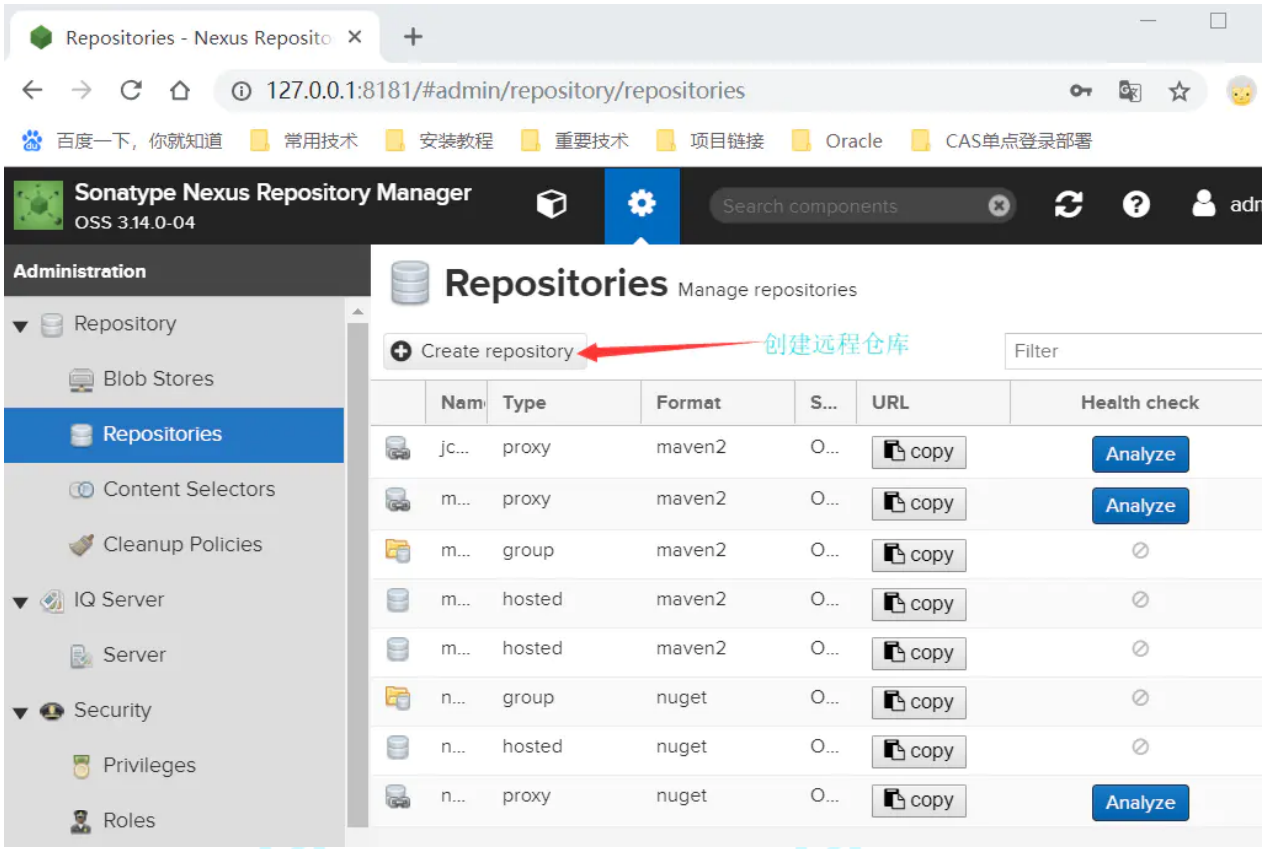


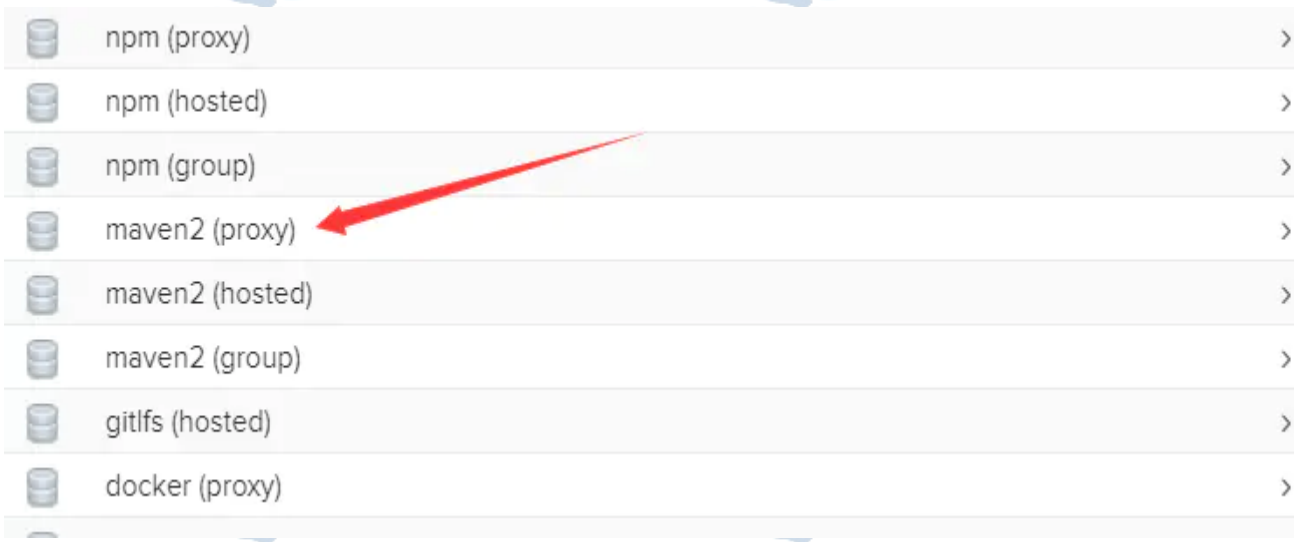
如上图所示，maven-public 就我创建的组仓库。以及还创建了 3 个代理仓库，如下。

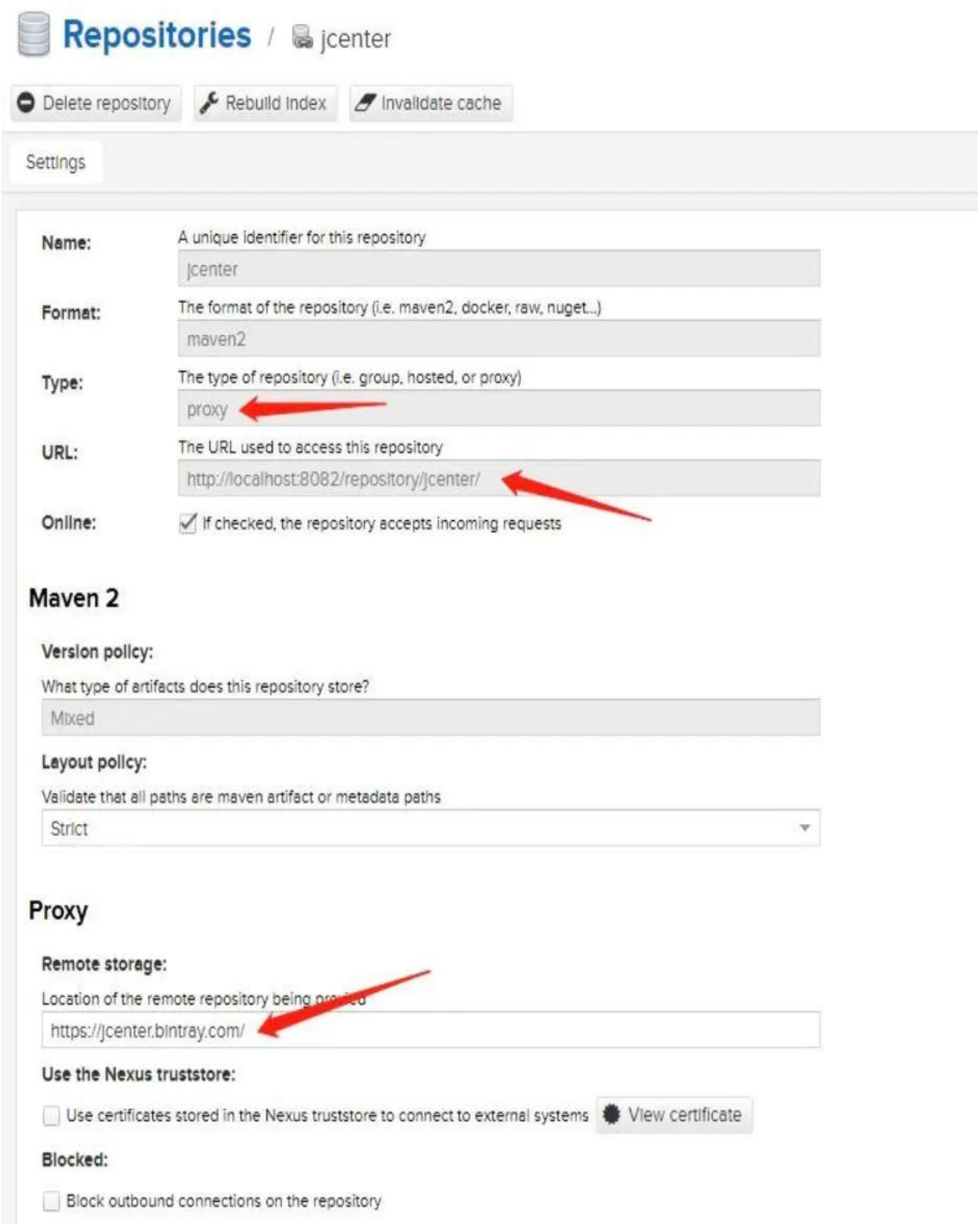
配置代理仓库：

1) jcenter 仓库：https://jcenter.bintray.com/

创建过程：（其余一样）

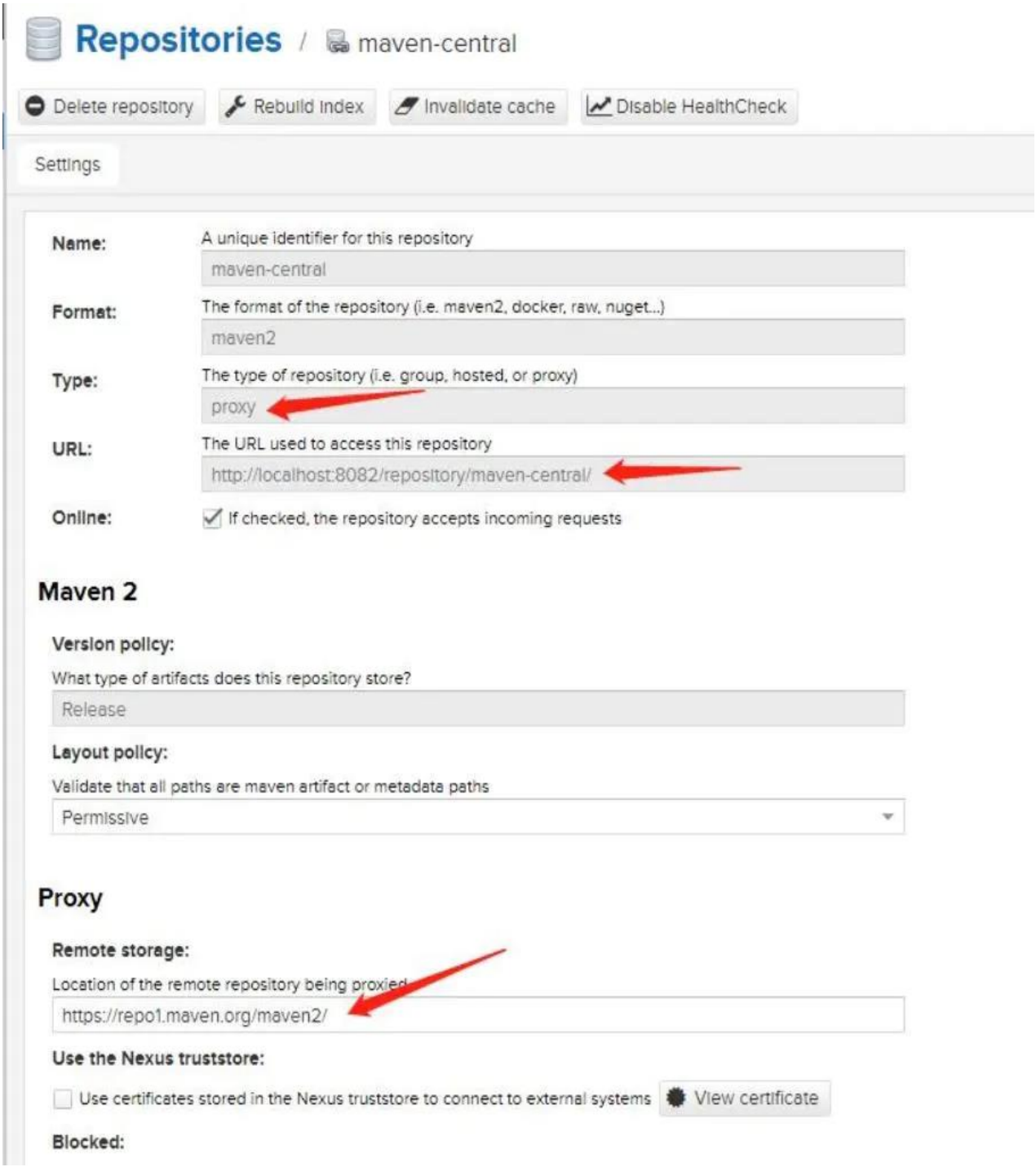

Repositories - Nexus Reposito
127.0.0.1:8181/#admin/repository/repositories
百度一下，你就知道
常用技术
安装教程
重要技术
项目链接
Oracle
CAS单点登录部署
Sonatype Nexus Repository Manager
OSS 3.14.0-04
Search components
Administration
Repository
Blob Stores
Repositories
Content Selectors
Cleanup Policies
IQ Server
Server
Security
Privileges
Roles
Repositories
Manage repositories
Create repository
创建远程仓库
Filter
Nam
Type
Format
S...
URL
Health check
jc...
proxy
maven2
O...
copy
Analyze
m...
proxy
maven2
O...
copy
Analyze
m...
group
maven2
O...
copy
m...
hosted
maven2
O...
copy
m...
hosted
maven2
O...
copy
n...
group
nuget
O...
copy
n...
hosted
nuget
O...
copy
n...
proxy
nuget
O...
copy
Analyze


npm (proxy)
npm (hosted)
npm (group)
maven2 (proxy)
maven2 (hosted)
maven2 (group)
gitlfs (hosted)
docker (proxy)


Repositories / jcenter
Delete repository
Rebuild Index
Invalidate cache
Settings
Name:
A unique identifier for this repository
jcenter
Format:
The format of the repository (i.e. maven2, docker, raw, nuget...)
maven2
Type:
The type of repository (i.e. group, hosted, or proxy)
proxy
URL:
The URL used to access this repository
http://localhost:8082/repository/jcenter/
Online:
If checked, the repository accepts incoming requests
Maven 2
Version policy:
What type of artifacts does this repository store?
Mixed
Layout policy:
Validate that all paths are maven artifact or metadata paths
Strict
Proxy
Remote storage:
Location of the remote repository being proxied
https://jcenter.bintray.com/
Use the Nexus truststore:
Use certificates stored in the Nexus truststore to connect to external systems
View certificate
Blocked:
Block outbound connections on the repository

2) maven 中央仓库：https://repo1.maven.org/maven2/

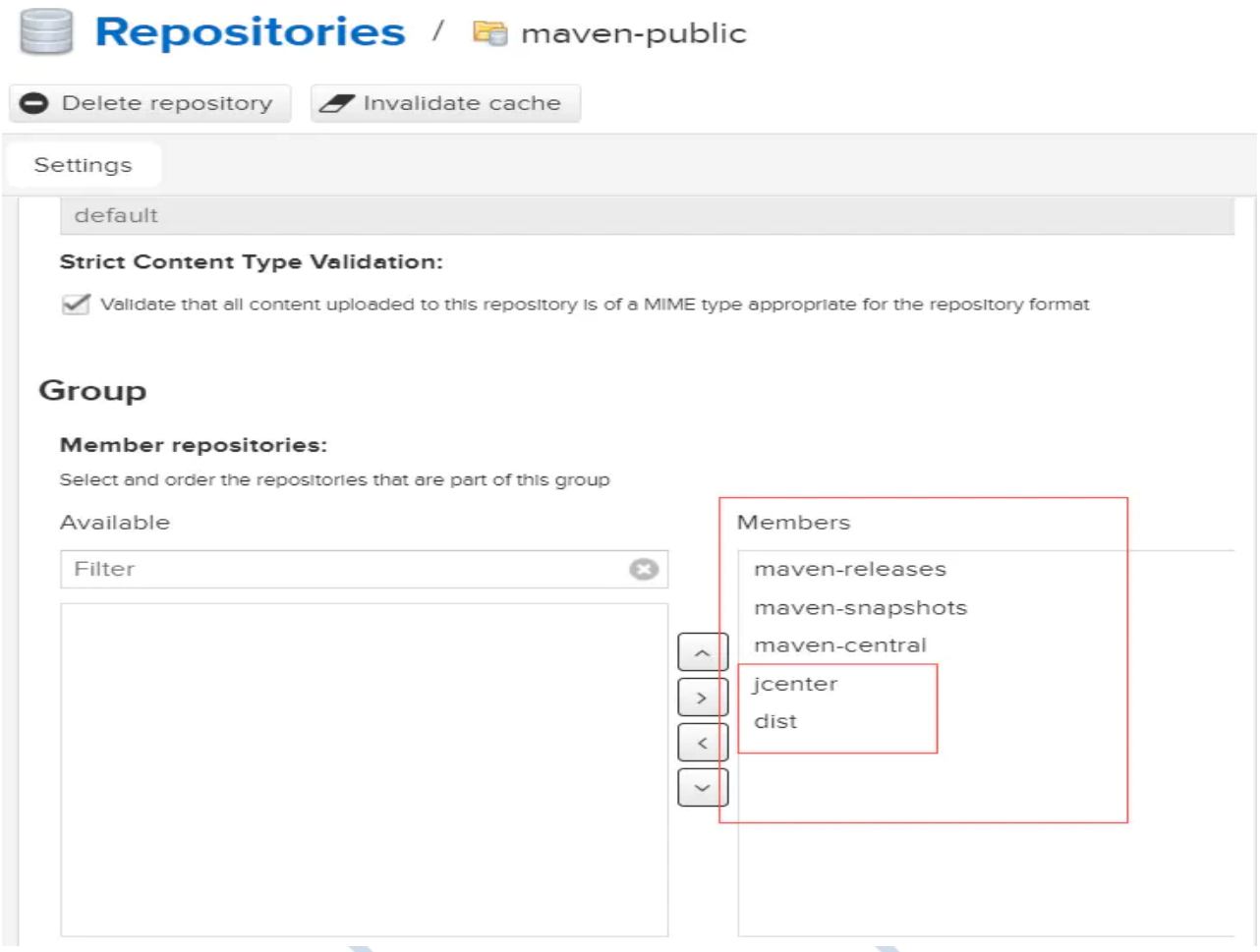
一般默认已经创建，没有创建就和上面一样 1）过程。



3) 公司内部 nexus 仓库，这里就不给出了 ---可以选择配置 创建公司的代理

4) 最后建立组仓库 maven-public，如下。

将创建的仓库添加进仓库组。



4 开发端配置

修改 maven 配置 setting.xml 文件

1. 创建好组仓库之后，修改 maven 配置 setting.xml 文件，添加 maven 仓库镜像，如下。Nexus 的仓库对于匿名用户只是只读的。为了能够部署构件，我们还需要再 settings.xml 中配置验证信息：

```
<server>
<id>nexus</id>
<username>admin</username>
<password>admin123</password>
</server>
```

配置远程仓库下载 jar 地址

```
<mirror>
<id>nexus</id>
<mirrorOf>*</mirrorOf>
<url>http://127.0.0.1:8181/repository/maven-public/</url>
</mirror>
```

接着修改 maven 项目中的 pom.xml，如下。

2. 不配置 setting.xml 中 server 就没有权限上传 jar，只能下载 jar。

   这个配置可以下载 jar 和部署构建

```
<!--============== 配置 nexus 私服 START ============== -->
<repositories>
  <repository>
    <id>maven-central</id>
    <name>maven-central</name>
```

```
    <!--仓库地址-->
    <url>http://127.0.0.1:8181/repository/maven-central/</url>
    <snapshots>
        <enabled>true</enabled>
    </snapshots>
    <releases>
        <enabled>true</enabled>
    </releases>
  </repository>
</repositories>

<pluginRepositories>
  <pluginRepository>
    <!--插件地址-->
      <id>nexus</id>
      <url>http://127.0.0.1:8181/repository/maven-public/</url>
  </pluginRepository>
 </pluginRepositories>

<distributionManagement>
  <repository>
      <id>nexus</id>
      <url>http://127.0.0.1:8181/repository/maven-releases/</url>
  </repository>
  <snapshotRepository>
     <!--上传快照-->
```

```
    <id>nexus</id>
    <name>Nexus Snapshot</name>
    <url>http://127.0.0.1:8181/repository/maven-snapshots/</url>
  </snapshotRepository>
  <site>
    <id>nexus</id>
    <name>Nexus Sites</name>
    <url>dav:http://127.0.0.1:8181/repository/maven-snapshots/</url>
  </site>
</distributionManagement>
<!--============== 配置私服 End ============== -->
```

部署构件到私服

（1） Nexus 的仓库对于匿名用户只是只读的。为了能够部署构件，我们还需要再 settings.xml 中配置验证信息：

```
<server>
  <id>snapshots</id>
  <username>admin</username>
  <password>admin123</password>
</server>
<server>
  <id>releases</id>
  <username>admin</username>
  <password>admin123</password>
</server>
```

其中，验证信息中 service 的 id 应该与 POM 中 repository 的 id 一致

（2）发布到私服的配置

```
<!-- 配置远程发布到私服，mvn deploy -->

<distributionManagement>

  <repository>

    <id>releases</id>

    <name> Nexus Release Repository </name>

    <url> http://127.0.0.1:8081/nexus/content/repositories/releases/ </url>

  </repository>

  <snapshotRepository>

    <id>snapshots</id>

    <name> Nexus Snapshot Repository </name>

    <url> http://127.0.0.1:8081/nexus/content/repositories/snapshots/ </url>
</snapshotRepository >

</distributionManagement >
```

部署命令：
还可以这么配

```
<distributionManagement>

   <repository>

     <id>maven-releases</id>

     <name>Nexus Release Repository</name>

     <url>http://localhost:8082/repository/maven-releases/</url>
```

```
    </repository>

    <snapshotRepository>

      <id>maven-snapshots</id>

      <name>Nexus Snapshot Repository</name>

      <url>http://localhost:8082/repository/maven-snapshots/</url>

    </snapshotRepository>

</distributionManagement>
```

如果是 gradle 项目，修改 init.gradle 文件，如下。

```
uploadArchives {

    def nexus_credentials = [userName: "admin", password: "admin123"]

    repositories.mavenDeployer {

        snapshotRepository(url: "http://127.0.0.1:8082/repository/maven-
snapshots/") {

            authentication(nexus_credentials)

        }

        repository(url: "http://127.0.0.1:8082/repository/maven-releases/") {

            authentication(nexus_credentials)

        }

    }

}
```

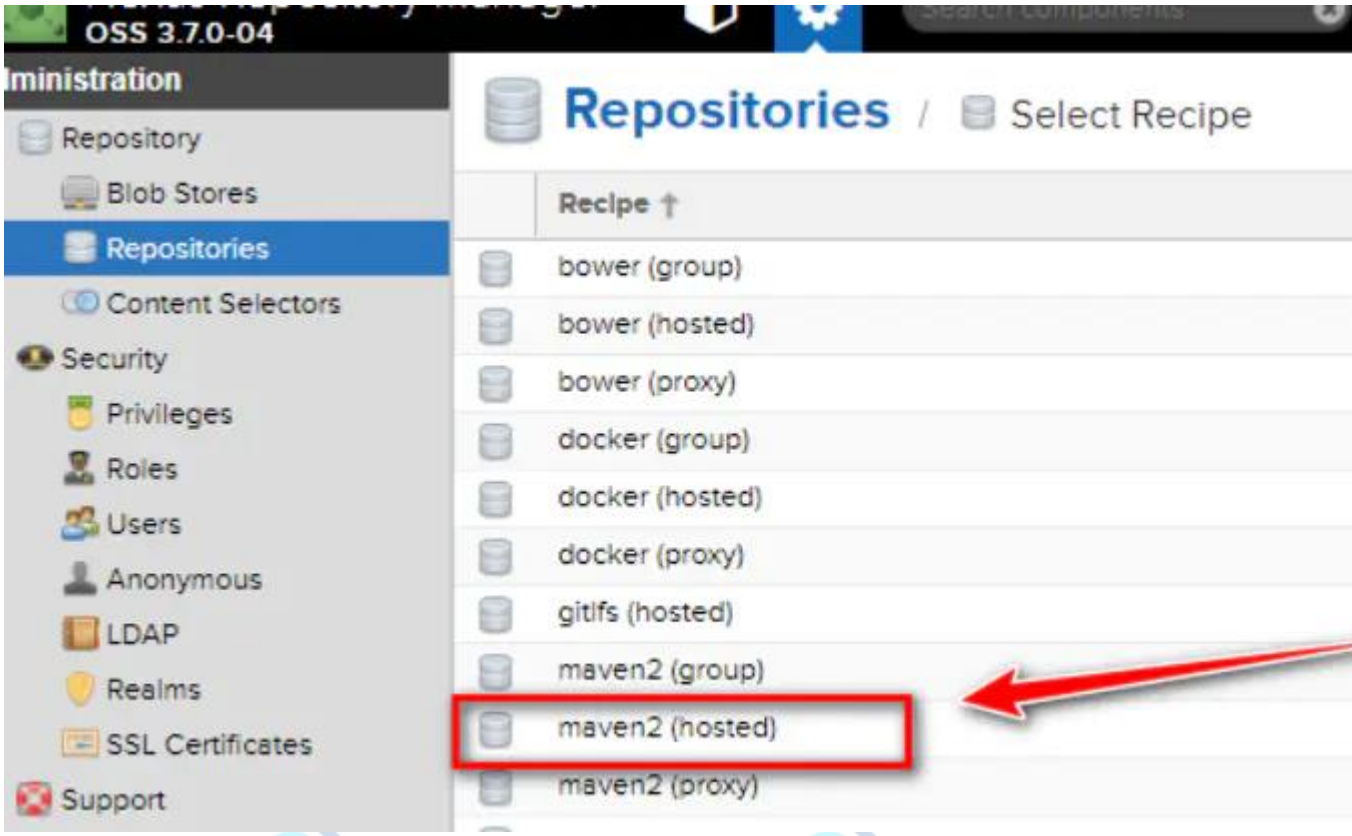
4.1 上传 jar 的两种方式：

比如上传这个 jar

nexus 提供了 3rd party、Snapshots、Releases 这三个目录存放第三方 jar 包

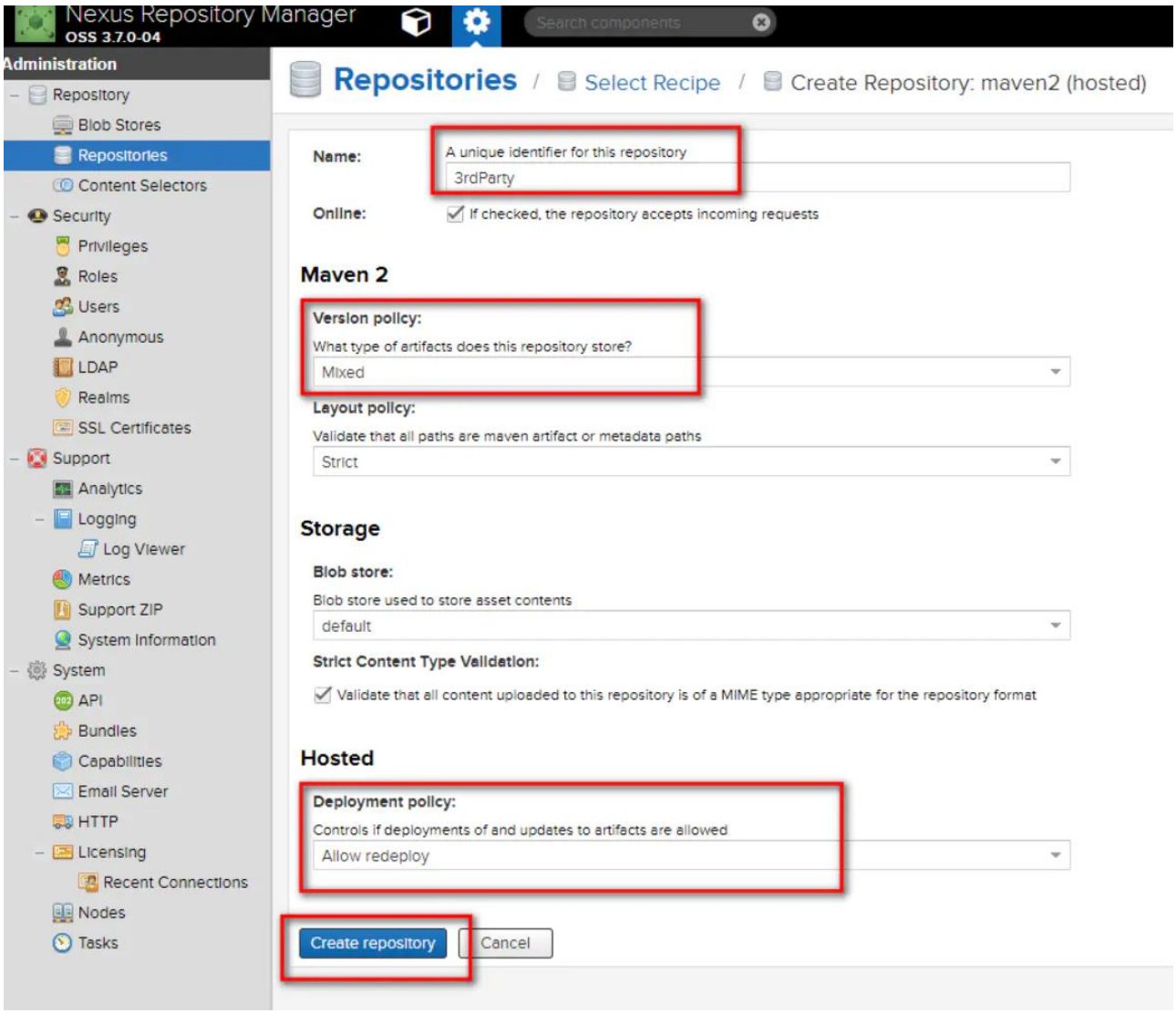
（1）登录 nexus（默认用户名密码/admin、admin123）

（2）Create repository



解释一下：

proxy：即你可以设置代理，设置了代理之后，在你的 nexus 中找不到的依赖就会去配置的代理的地址中找 hosted：你可以上传你自己的项目到这里面 group：它可以包含前面两个，是一个聚合体。一般用来给客户一个访问 nexus 的统一地址。 简单的说，就是你可以上传私有的项目到 hosted，以及配置 proxy 以获取第三方的依赖（比如可以配置中央仓库的地址）。前面两个都 弄好了之后，在通过 group 聚合给客户提供统一的访问地址。



修改 maven 安装目录下的 /conf/settings.xml 文件，添加 server 节点。如图：

4.1.1 1.方式一

开始上传 jar

打开 cmd 输入修改后的模板

模板

```
mvn deploy:deploy-file -DgroupId=xxx.xxx -DartifactId=xxx -Dversion=0.0.2 -
Dpackaging=jar -Dfile=D:\xxx.jar -
Durl=http://xxx.xxx.xxx.xxx:8081/repository/3rdParty/ -DrepositoryId=3rdParty
```

模板解析：

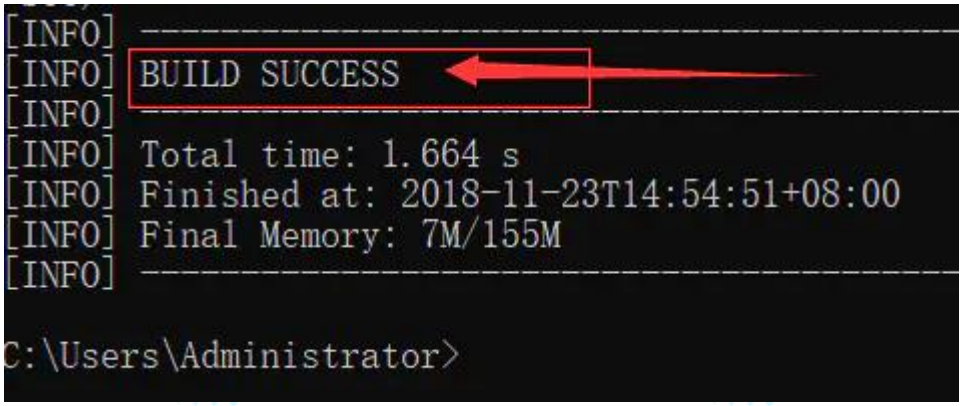
-DgroupId 为上传的 jar 的 groupId-DartifactId 为上传的 jar 的 artifactId-Dversion 为上传的 jar 的需要被依赖的时候的版本号-Dpackaging 为 jar，-Dfile 为 jar 包路径-Durl 为要上传的路径，可以通过以下方式获取到

举个例子：

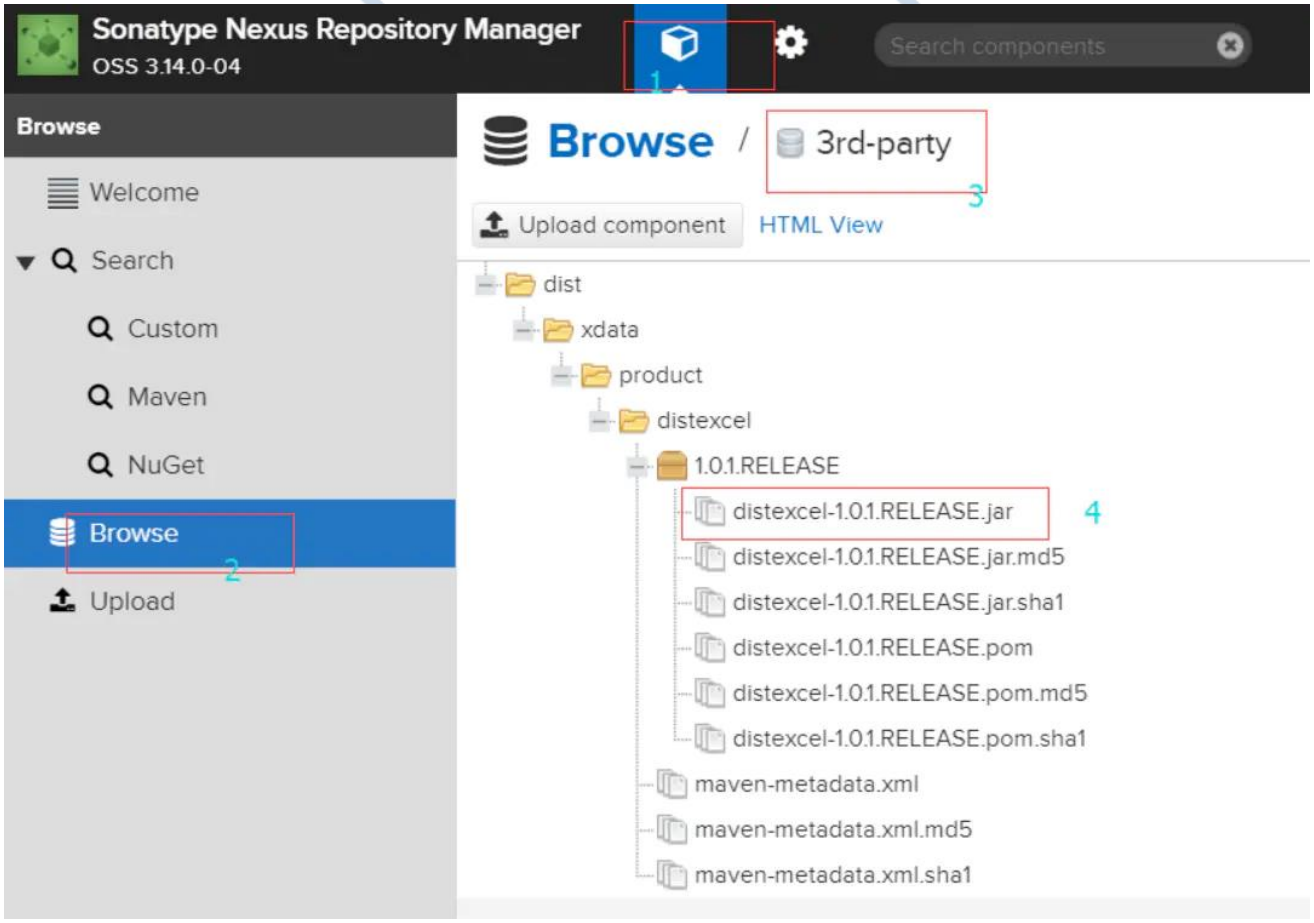
```
C:\Users\Administrator>mvn deploy:deploy-file -DgroupId=dist.xdata.product -
DartifactId=distexcel -Dversion=1.0.1.RELEASE -Dpackaging=jar -
Dfile=I:\maven\repository\dist\xdata\product\distexcel\1.0.1.RELEASE\distexcel-
1.0.1.RELEASE.jar -Durl=http://192.168.2.81:8181/repository/3rd-party/ -
DrepositoryId=3rd-party
```

```
mvn deploy:deploy-file -DgroupId=dist.xdata.product -DartifactId=distexcel -
Dversion=1.0.1.RELEASE -Dpackaging=jar -
Dfile=I:\maven\repository\dist\xdata\product\distexcel\1.0.1.RELEASE\distexcel-
1.0.1.RELEASE.jar -Durl=http://192.168.2.81:8181/repository/3rd-party/ -
DrepositoryId=3rd-party
```

上传成功：



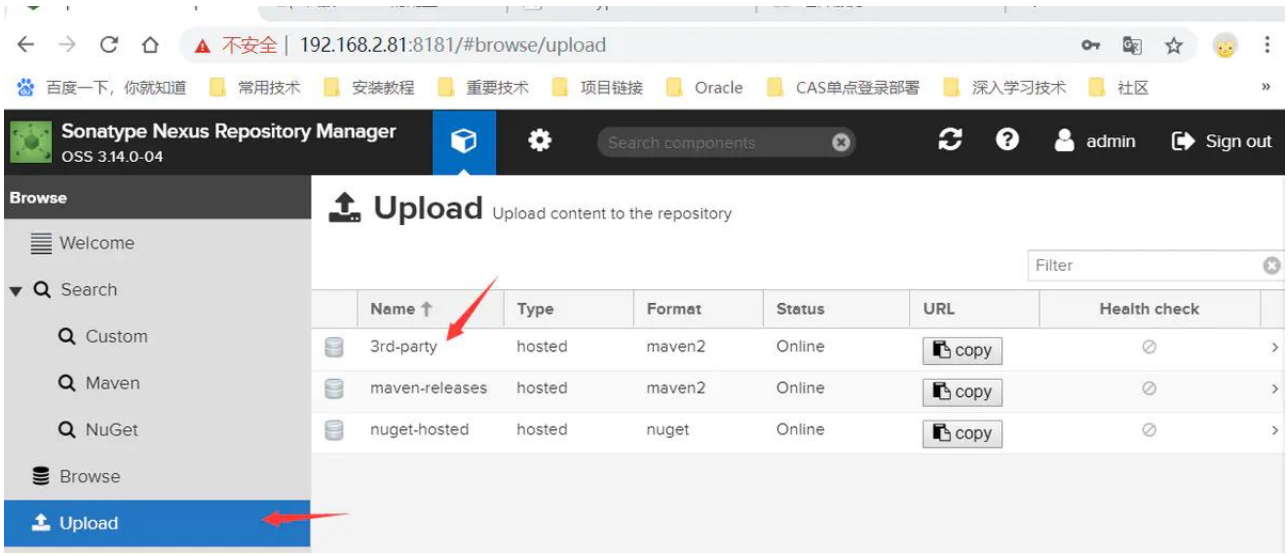
上传后在 nexus 中查看：

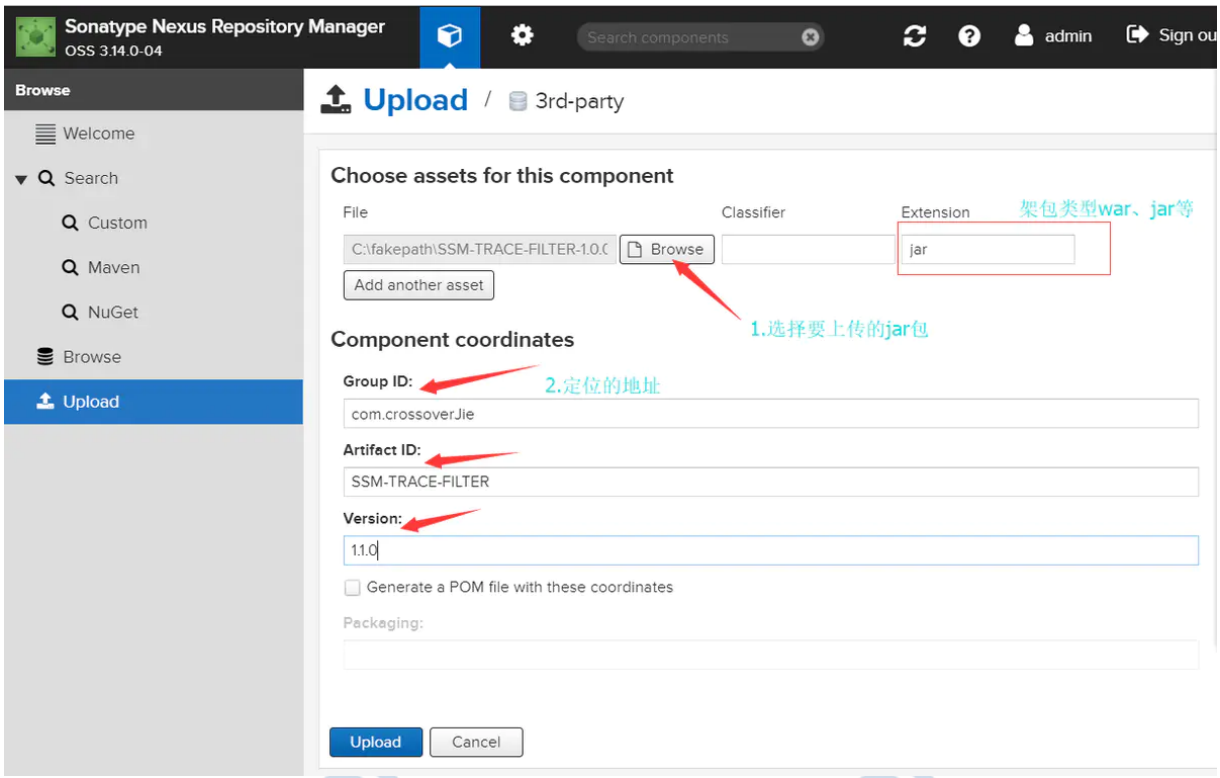


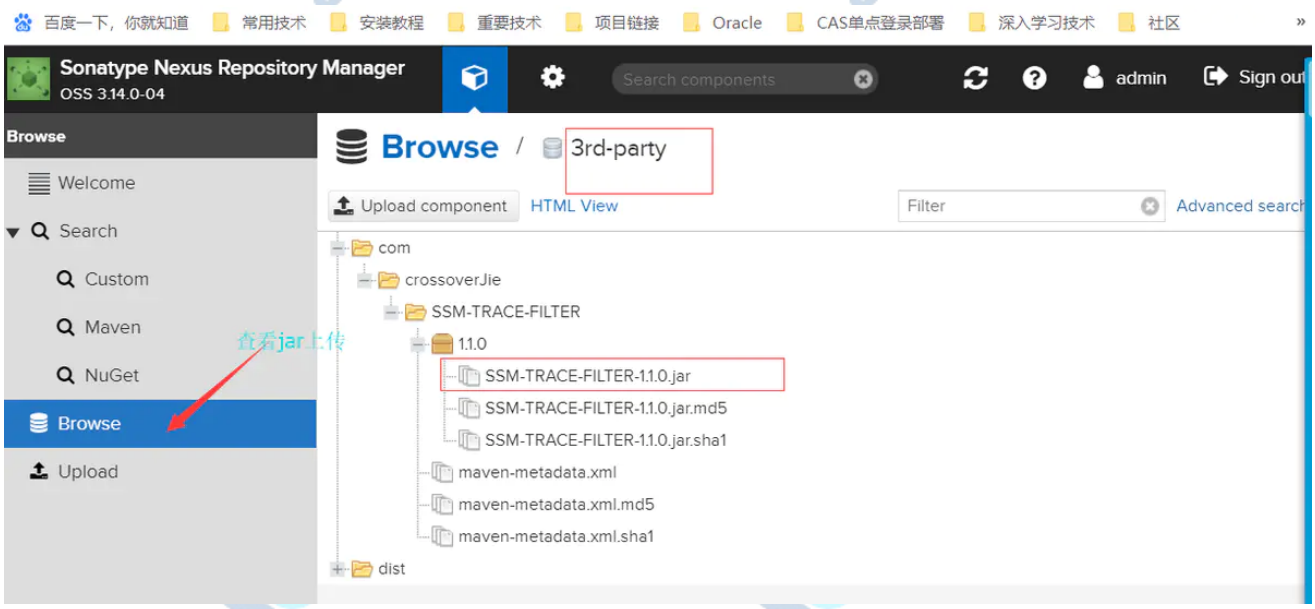
4.1.2 上传方式二

更简单的上传方式

nexus 提供了 3rd party、Snapshots、Releases 这三个目录存放第三方 jar 包



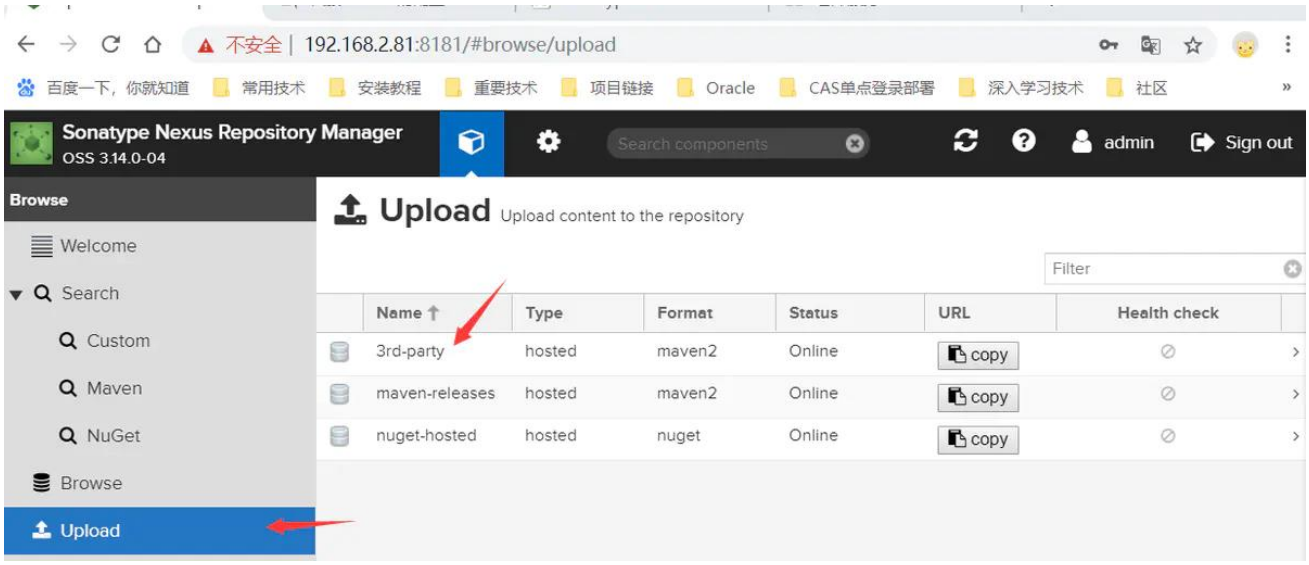


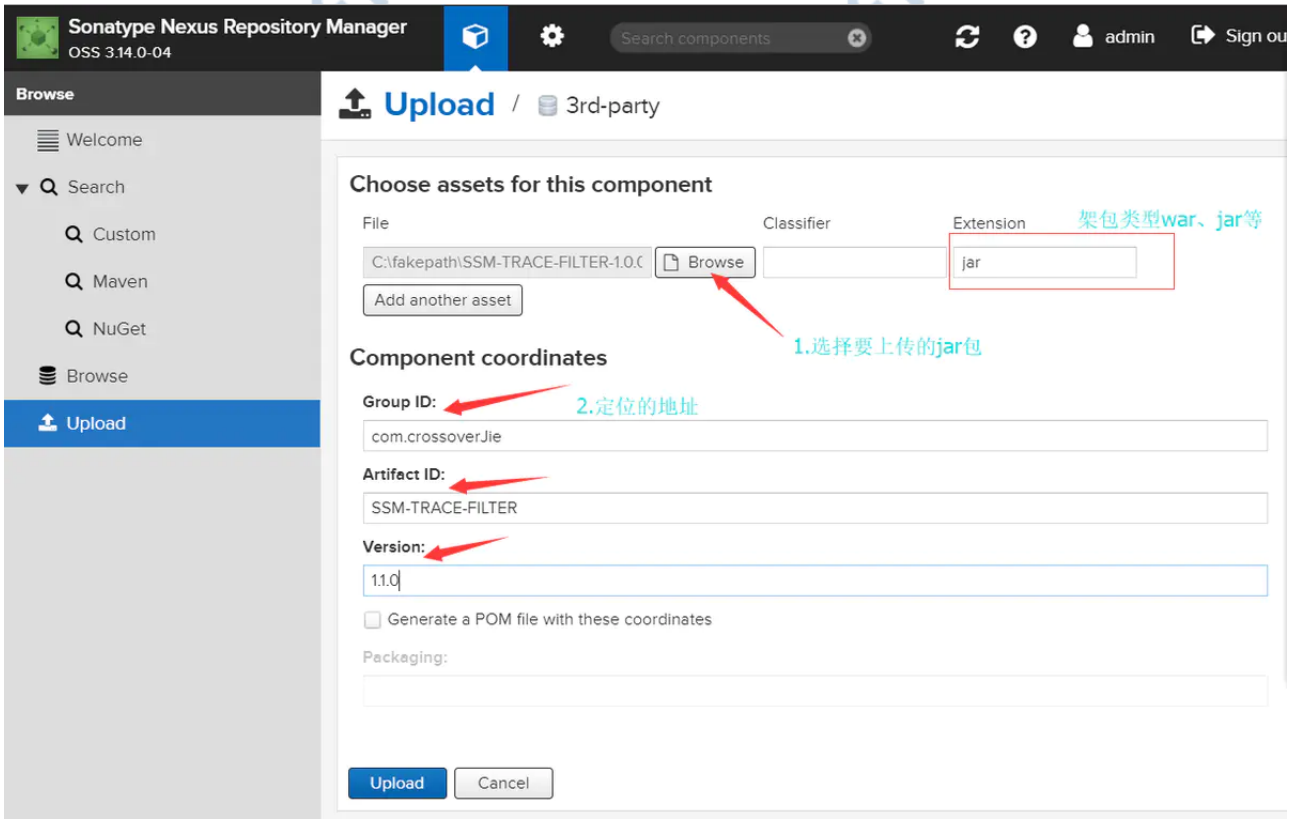


2.上传方式二：

更简单的上传方式

nexus 提供了 3rd party、Snapshots、Releases 这三个目录存放第三方 jar 包

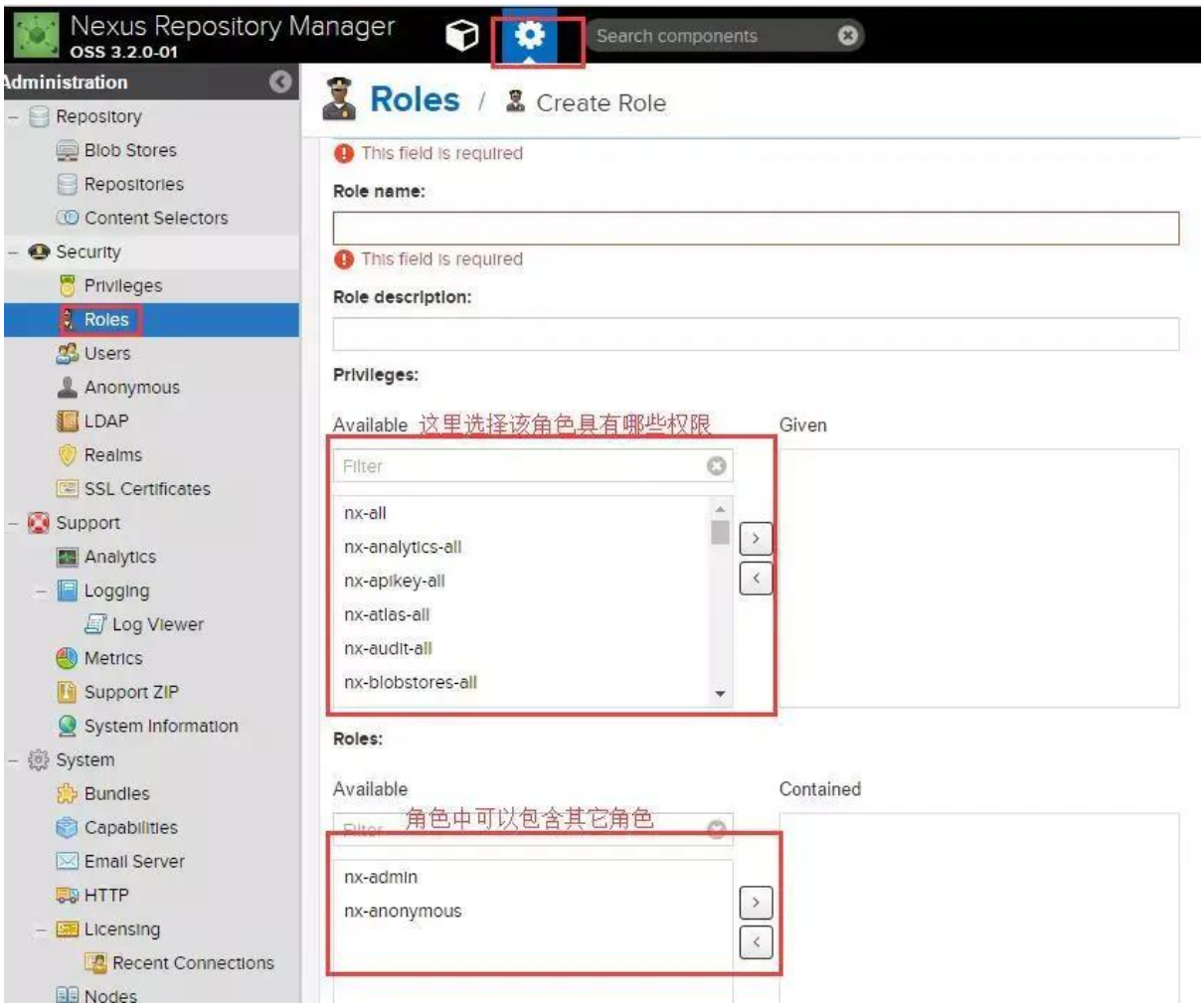




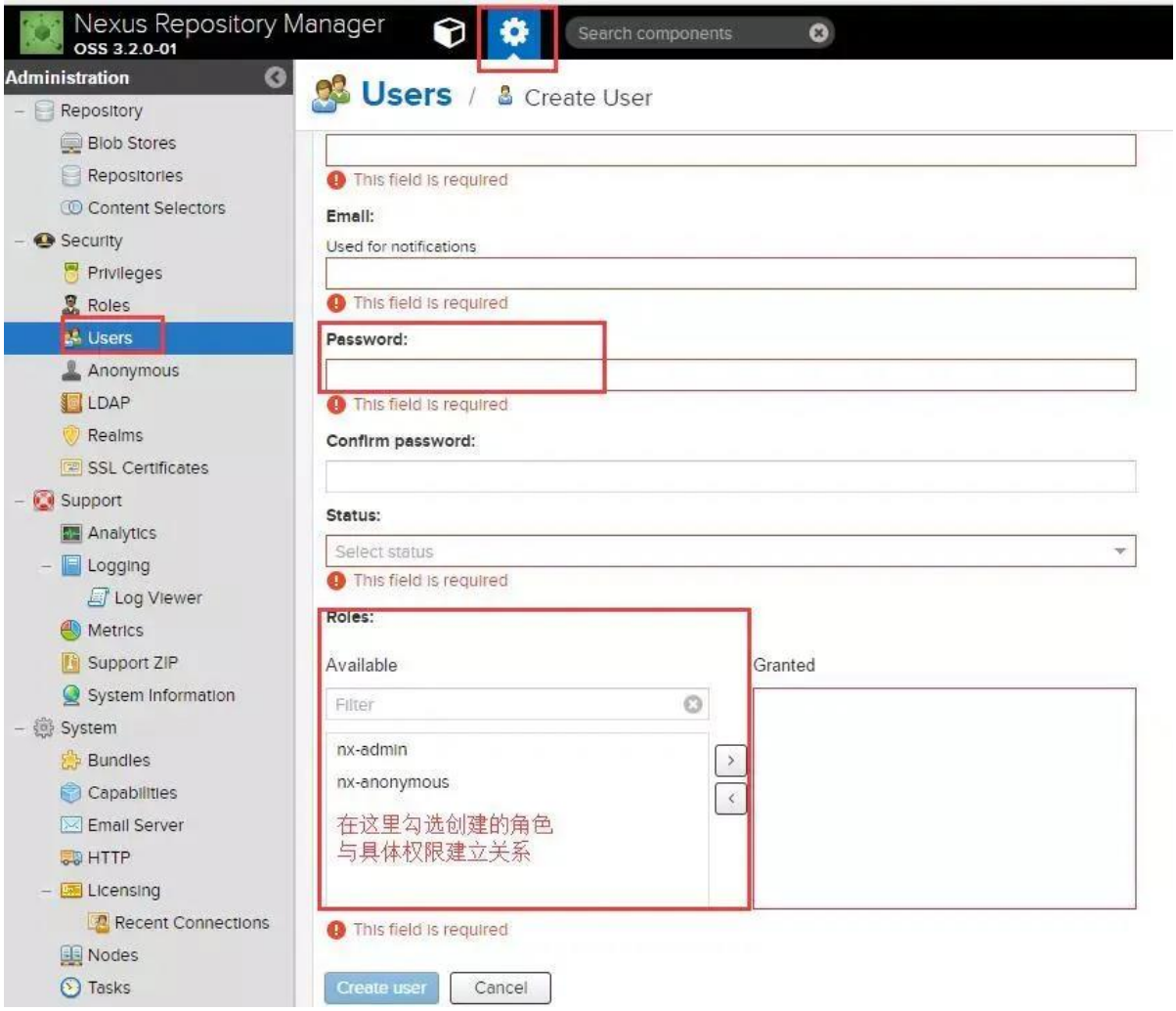
4.2 nexus3.x 权限配置

1. 去掉“勾”是禁用匿名访权限。

2. 角色创建


Nexus Repository Manager
OSS 3.2.0-01
Search components
Administration
Repository
Blob Stores
Repositories
Content Selectors
Security
Privileges
Roles
Users
Anonymous
LDAP
Realms
SSL Certificates
Support
Analytics
Logging
Log Viewer
Metrics
Support ZIP
System Information
System
Bundles
Capabilities
Email Server
HTTP
Licensing
Recent Connections
Nodes
Roles / Create Role
This field is required
Role name:
This field is required
Role description:
Privileges:
Available 这里选择该角色具有哪些权限
Given
Filter
nx-all
nx-analytics-all
nx-apikey-all
nx-atlas-all
nx-audit-all
nx-blobstores-all
Roles:
Available
Contained
角色中可以包含其它角色
nx-admin
nx-anonymous

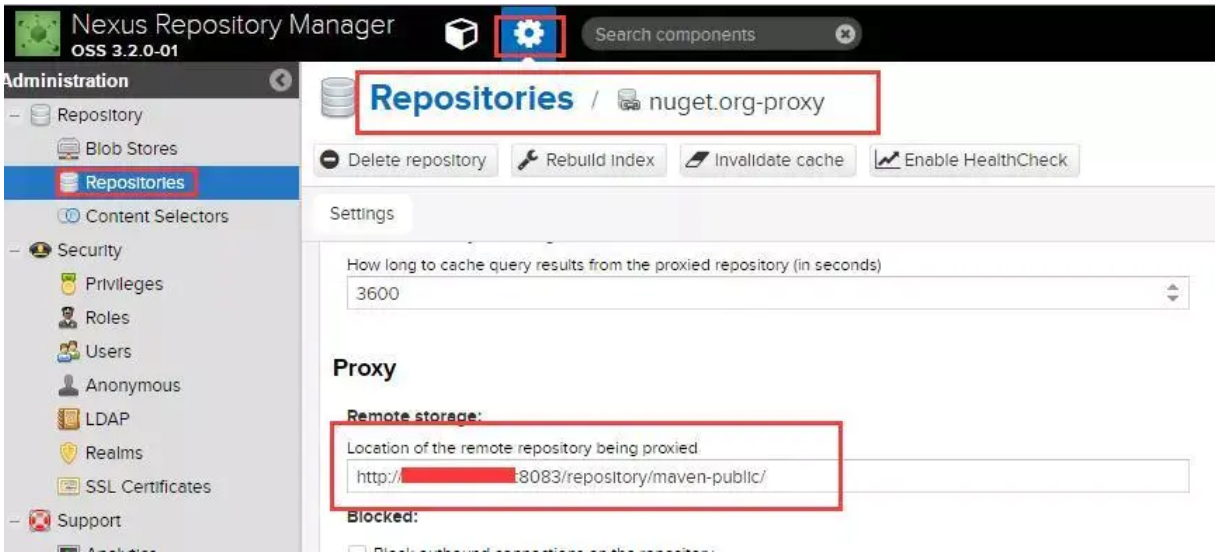
3. 人员创建



4.代理配置

通过该配置可以使得两个 Nexus 服务器相关联。

（1） 配置地址



配置用户

4.3 配置 Rsync（服务端）

rsyncd.conf

```
port = 873 # 这个是默认端口可以修改的

read only = true # 只允许读

transfer logging = false
```

```
log format = %t %a %m %f %b

log file = /var/log/rsync.log

hosts allow = *

secrets file = /etc/rsyncd/rsyncd.secrets

motd file = /etc/rsyncd/rsyncd.motd

pid file = /var/run/rsyncd.pid

use chroot = true

uid = root

gid = root

[nexus]

comment = Nexus Rsync

auth users = root

ignore errors

list = no

path = /datahub/nexus3_volumes/data/blobs
```

rsyncd.secrets 冒号后面的密码是举例子，请设置复杂些

root:123456

rsyncd.motd

```
############################################################

Nexus Rsync

############################################################
```

4.4 进行资源同步（客户端获取服务端）

为了保持数据的一致性，我完全以家里的为准。

```
/usr/bin/rsync -avr --delete --port=873 --password-file=/home/rsync.pwd
root@xxx.xxx.xxx.xxx::nexus /data/nexus3_volumes/data/blobs
```

也可以设置一个定时任务，在什么时候自动进行数据的获取。我这里设置的是每天凌晨 12 点半进行获取。

```
30 0 * * * /usr/bin/rsync -avr --delete --port=873 --password-file=/home/rsync.pwd
root@xxx.xxx.xxx.xxx::nexus /data/nexus3_volumes/data/blobs > /dev/null 2>&1
```

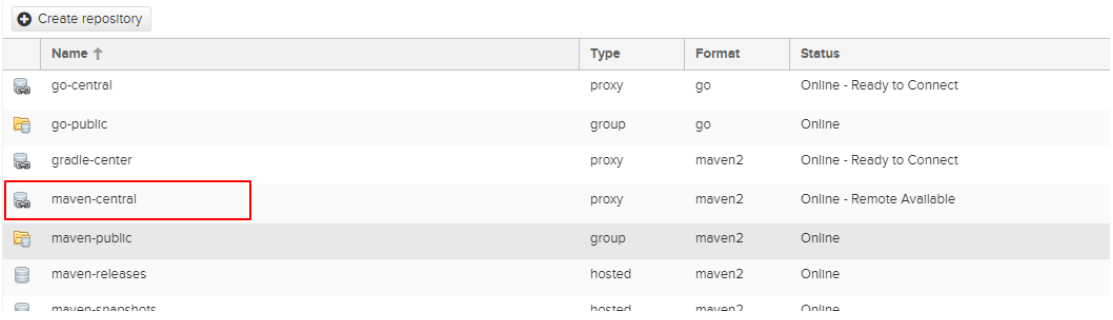
至此，我们就可以把家里产生的依赖文件都同步到公司的机器上了，省去了从远程重新获取的步骤。

4.5 重建索引

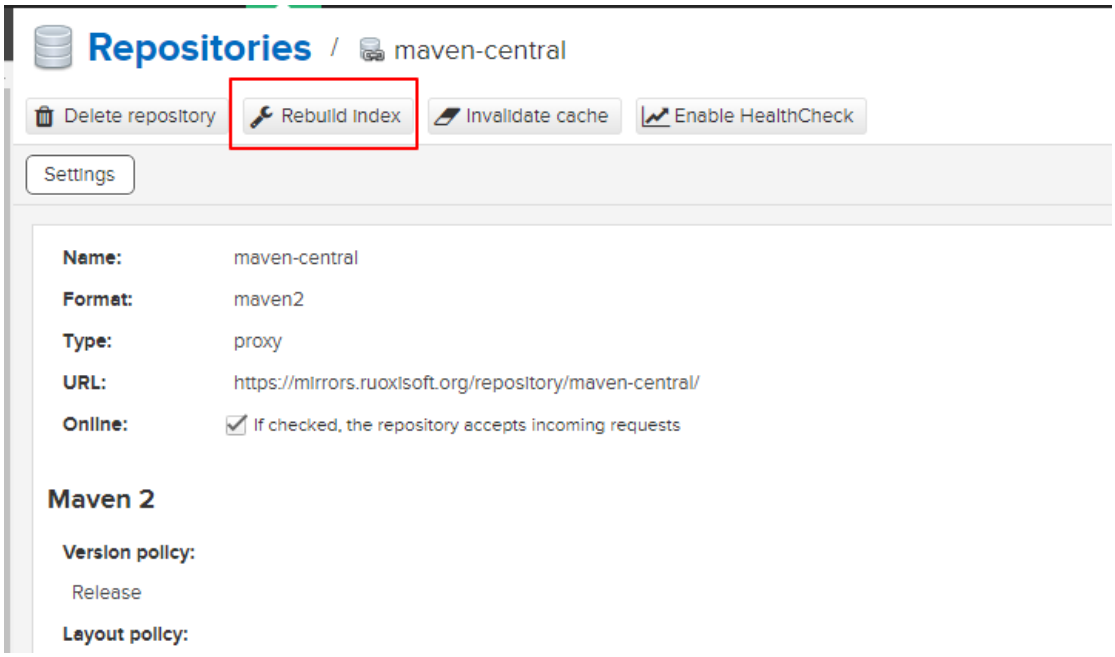
文件同步完成后，需要重新赋予下权限以及重建下索引。

```
chown -R 200:200 /data/nexus3_volumes/data/blobs # 后面的路径是我公司环境的路径
```

然后进入到 nexus3 重建下索引



选择其中的一个（暂时没有找到批量的方法）



然后 rebuild index

重启 nexus3，如果不这么做的话，即使同步了还是找不到相应的包。

4.6 手动下载依赖包

我们可以使用 mvn 手动下载依赖包

4.6.1 创建 pom

```
<?xml version="1.0" encoding="UTF-8"?>

<project xmlns="http://maven.apache.org/POM/4.0.0"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"

xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
https://maven.apache.org/xsd/maven-4.0.0.xsd">

<modelVersion>4.0.0</modelVersion>

<groupId>test</groupId>
```

```
<artifactId>test</artifactId>

<version>0.0.1-SNAPSHOT</version>

<dependencies>

<!-- https://mvnrepository.com/artifact/mysql/mysql-connector-java -->

<dependency>

<groupId>mysql</groupId>

<artifactId>mysql-connector-java</artifactId>

<version>8.0.20</version>

</dependency>

</dependencies>

</project>
```

4.6.2 执行下载命令

```
mvn -f offline_pom.xml dependency:tree # 下载依赖 树形式展示依赖

mvn -f offline_pom.xml dependency:list # 下载依赖 列表形式展示依赖

mvn -f offline_pom.xml dependency:sources # 下载 sources

mvn -f offline_pom.xml dependency:resolve -Dclassifier=javadoc # 下载 documentation

mvn -f offline_pom.xml dependency:copy-dependencies # 下载依赖，并复制依赖到当前文件夹
```

这样即使家里未运行项目，也可以获取依赖了。